# LLM Agents – Planning and Reasoning

Autonomous Software Agents

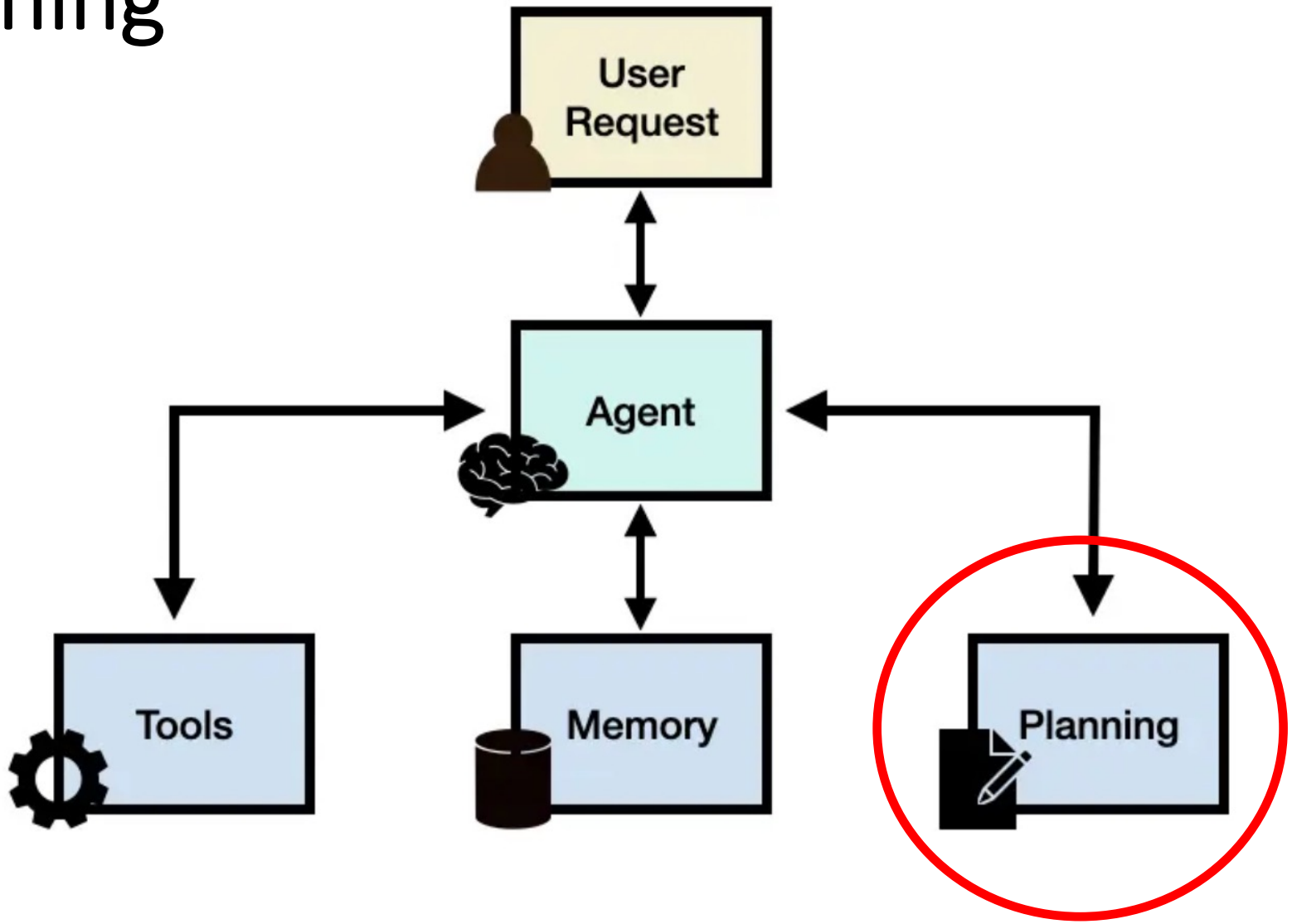
A.A. 2025-2026

**Prof. Paolo Giorgini**



UNIVERSITY OF TRENTO - Italy
Department of Information and Communication Technology

# Planning



# Planning

- Planning means reasoning about **how to reach the goal while respecting constraints**
  - Eg.  Goal: plan a two-week trip from San Francisco to India / Constraint: budget of $5,000
- Effective planning requires the agent to:
  - Understand the goal
  - Consider **multiple possible approaches**
  - Choose the **most promising one**
- Not all decompositions lead to:
  - a successful outcome or an efficient solution
    - Eg : “How many companies without revenue have raised at least $1 billion?”
    - Two possible plans:
      - Find companies without revenue → then filter by funding
      - Find companies with ≥ $1B funding → then filter by revenue
    - **Plan 2 is more efficient** and should be preferred

# Why Use Natural Language for Planning

- Using **natural language plans**:
  - Makes the planner **robust to changes in tool APIs**
  - Reduces the need to update prompts and examples when tools change
  - Makes planners easier to reuse across different tool inventories
- Natural language plans are **higher-level** than domain-specific function names
- Natural language actions must be **translated** into executable commands
- Translation is:
  - Simpler than planning
  - Less prone to hallucination
  - Can be handled by weaker models

```
1. get current date
2. retrieve the best-selling product last week
3. retrieve product information
4. generate query
5. generate response
```

# Coupling vs Decoupling Planning and Execution

- Planning and execution can be:
  - **Coupled** (plan and execute in one prompt)
  - **Decoupled** (plan first, execute later)
- Problem with coupling:
  - The agent may generate a very long plan
  - The plan may be inefficient or incorrect
  - The agent may waste time and API calls

```
User: What's the weather in Trento?

LLM:
Thought: I should search for weather
Action: search("weather Trento")
Observation: 18°C
Answer: It's 18°C in Trento
```

**Step 1 – Planning**

```
1. Search current weather in Trento
2. Extract temperature
3. Return result
```

**Step 2 – Execution**

- Key principle: **Planning should be decoupled from execution**
  - you can review the plan, correct it before execution, and operate more efficiently

# Plan Validation and Heuristics

- Before execution, a plan should be **validated**
- Simple validation heuristics include:
  - Eliminating plans with **invalid actions**
    - e.g., requiring Google Search when the agent has no access to it
  - Eliminating plans with **too many steps**
- Plans can also be validated using **AI judges**
  - Evaluate whether the plan is reasonable
  - Suggest improvements
- If the plan is bad → generate a new plan

# An Example

*“Project the sales revenue for Fruity Fedora over the next three months”*

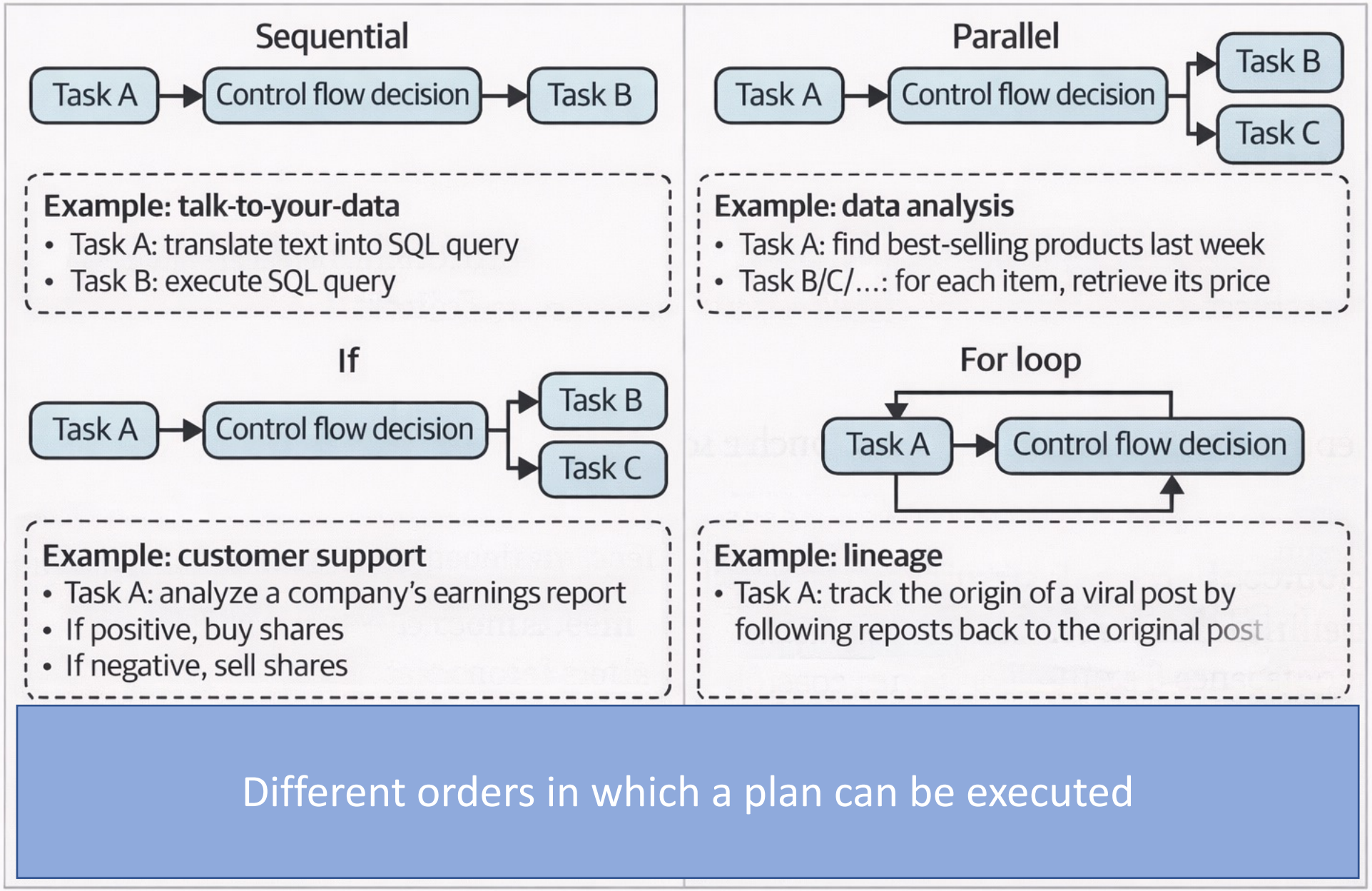
1. Reason about the task
   - Decide what information is needed (e.g. historical sales)
2. Generate SQL queries
   - Request sales data from the last several years
3. Execute SQL queries
   - Retrieve results from the database
4. Reason over results
   - Check data quality (e.g. missing values)
5. Decide to gather more data
   - Identify need for marketing campaign information
6. Generate and execute additional SQL queries
7. Integrate all information
   - Produce a sales projection
8. Verify completion
   - conclude that the task has been successfully accomplished

# Complex Plans and Control Flow

- So far, most plans have been **sequential**:
  - Each action is executed **after** the previous one completes
  - The order in which actions are executed is called **control flow**
- **Sequential control flow** is only one possible form
  - **Parallel:** Execute multiple tasks **at the same time**
  - **If (conditional):** Execute different actions based on a condition
  - **For loop (iteration):** Repeat a task until a condition is met
- Control flow determines **how a plan is executed**, not just *what* actions are taken

# Key Observations

- In traditional software, control flow conditions are **explicit and exact**
- In AI-powered agents:
  - Control flow decisions are made by the **model**
- Non-sequential plans are:
  - Harder to generate
  - Harder to translate into executable commands
- When evaluating an agent framework, it is important to check:
  - Which control flows are supported
  - Whether parallel execution is available


Sequential
Task A
Control flow decision
Task B
Example: talk-to-your-data
• Task A: translate text into SQL query
• Task B: execute SQL query
Parallel
Task A
Control flow decision
Task B
Task C
Example: data analysis
• Task A: find best-selling products last week
• Task B/C/...: for each item, retrieve its price
If
Task A
Control flow decision
Task B
Task C
Example: customer support
• Task A: analyze a company's earnings report
• If positive, buy shares
• If negative, sell shares
For loop
Task A
Control flow decision
Example: lineage
• Task A: track the origin of a viral post by following reposts back to the original post
Different orders in which a plan can be executed

# Execution, Reflection, and Error Correction

Solving a task typically involves the following loop:

1. **Plan generation:** decompose the task into manageable steps
2. **Reflection & plan evaluation:** decide whether the plan is good
3. **Execution:** execute actions (often via function calls / tools)
4. **Reflection & error correction:** evaluate outcomes and fix mistakes or generate a new plan if needed

- Reflection is not mandatory, but it **significantly boosts performance**

## Symbolic Planners

- Use **explicit symbolic representations**: states, actions, preconditions, effects
- Rely on **formal models of the environment**
- Correct and interpretable plans
- Limitations:
  - Require **manual modeling**
  - Poor scalability to open-ended or uncertain environments
  - Fragile when assumptions are violated

## RL Planners

- Learn planning implicitly through **interaction and reward optimization**
- Planner is trained via an **RL algorithm**
- Strengths:
  - Good for sequential decision-making
  - Can handle stochastic environments
- Limitations:
  - **Expensive training**
  - Low interpretability
  - Hard to generalize beyond training distribution

## FM Planners

- The **model itself acts as the planner**
- Planning emerges frm pretraining, prompting, tool use
- Strengths:
  - No explicit world model required
  - Flexible and domain-agnostic
  - Easy to adapt to new tasks
- Limitations:
  - No formal guarantees
  - Can generate inefficient or invalid plans

# Remarks on Planning

- Symbolic planners → **formal correctness**
- RL planners → **optimal policies via learning**
- FM planners → **flexible reasoning and decomposition**
- Likely future:
  - **Hybrid systems**
  - FM planners augmented with:
    - symbolic constraints
    - RL-based optimization
- Planning paradigms are **converging rather than competing**

# Can Foundation Models Plan? The Debate

- An **open question** is how well foundation models can perform planning
- Many researchers argue that **autoregressive LLMs cannot truly plan**
- Notable positions:
  - **Yann LeCun (2023)**: autoregressive LLMs *cannot plan*
  - **Kambhampati (2023)**:
    - LLMs are strong at **knowledge extraction**
    - But often confuse **plausible-looking reasoning** with **executable plans**
- Plans generated by LLMs may:
  - Appear reasonable to humans
  - Lead to **execution-time failures and errors**
- Still unclear whether:
  - LLMs fundamentally cannot plan, or
  - We have not yet learned how to use them properly for planning

# Planning as Search and the Role of LLMs

**Planning is fundamentally a search problem**

- Explore multiple paths to a goal
- Predict the outcome (reward) of each path
- Select the most promising one

Search often requires **backtracking**

- Trying one action
- Abandoning it if it leads to a poor state

- Criticism of autoregressive models:
  - They generate actions sequentially
  - Supposedly cannot backtrack
- Counter-argument:
  - An LLM can revise a plan after evaluating outcomes
  - Or restart and choose a different path

- Recent work suggests:
  - LLMs may implicitly model the world well enough to **predict outcomes**
- Effective planning may require augmenting LLMs with:
  - Search tools
  - State tracking systems
  - Even if LLMs cannot plan alone, they can still be a **core component of a planner**

# Plan Generation via Prompt Engineering

- The agent is instructed to:
  - Generate a **plan**, not the final answer
  - Use only a **fixed set of valid actions**
- The plan must be a **sequence of valid actions e**xecutable by the agent

```
Propose a plan to solve the task.
You have access to 5 actions:

get_today_date()
fetch_top_products(start_date, end_date, num_products)
fetch_product_info(product_name)
generate_query(task_history, tool_output)
generate_response(query)

The plan must be a sequence of valid actions.
```

# An Example

```
Task: "What's the price of the best-selling product last week?"
```

**Generated Plan**

```
1. get_today_date()
2. fetch_top_products()
3. fetch_product_info()
4. generate_query()
5. generate_response()
```

Parameters are **not fixed in advance**
They are inferred from:

- Previous tool outputs
- Task history

```
fetch_top_products(
  start_date = "2030-09-07",
  end_date   = "2030-09-13",
  num_products = 1
)
```

**Key Observation**

- Both the **action sequence** and the **parameters** are generated by the model
- This can lead to:
  - Hallucinated actions
  - Valid actions with wrong parameters

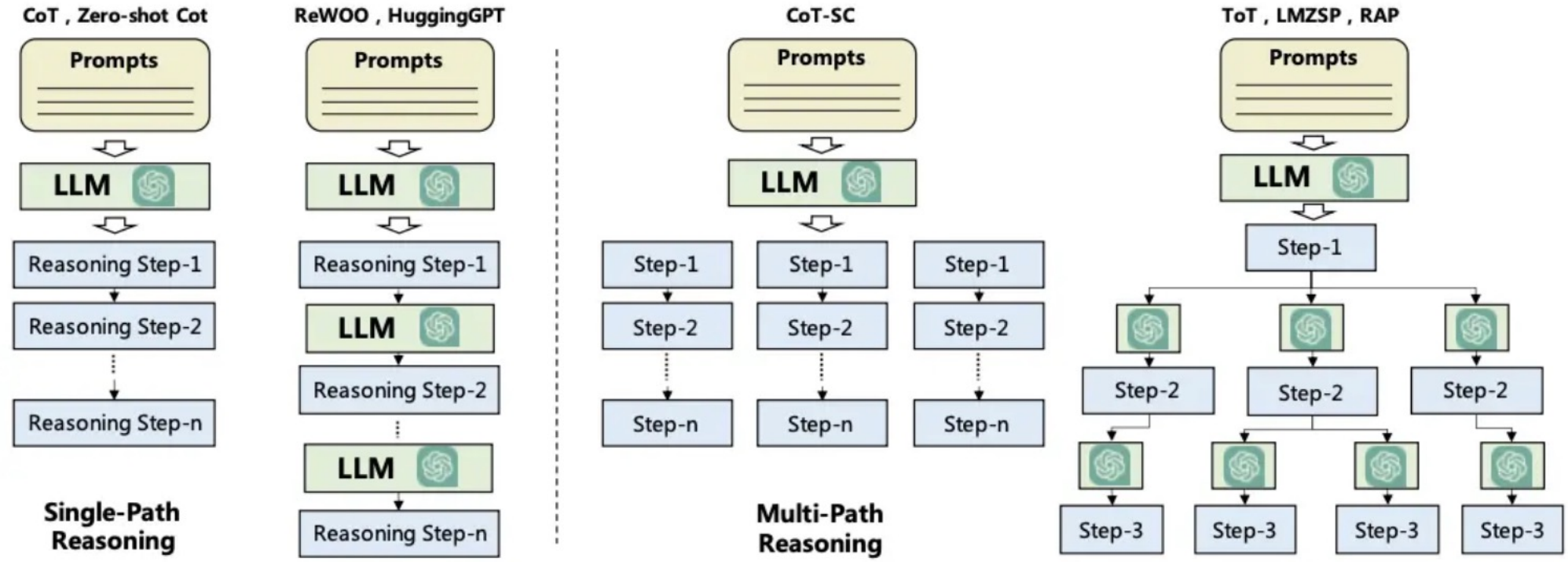
# Planning Granularity

- Plans can exist at **different levels of granularity**:
  - High-level (e.g., year-by-year, quarter-by-quarter)
  - Medium-level (e.g., month-by-month)
  - Low-level (e.g., week-by-week or exact function calls)
- **Planning vs Execution Trade-off**
  - **Detailed plans**:
    - Harder to generate
    - Easier to execute
  - **High-level plans**:
    - Easier to generate
    - Harder to execute
- A common solution is **hierarchical planning**:
  - First generate a high-level plan
  - Then refine each step into more detailed sub-plans

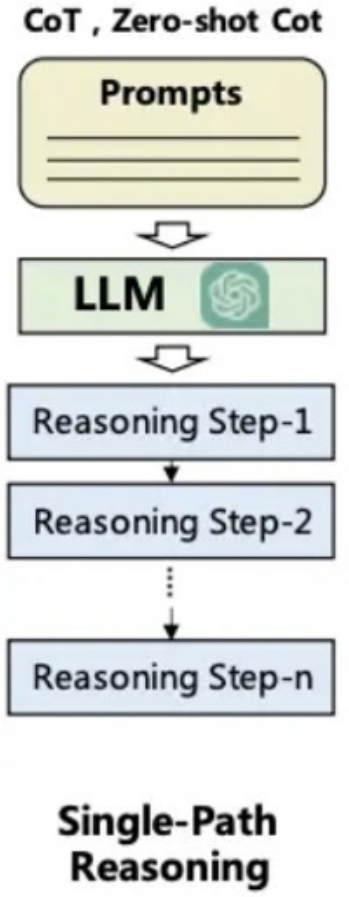
# Planning Module – without feedback

- Breaks down necessary steps or subtasks to answer user requests
- Helps the agent reason better and find solutions reliably
- Uses an LLM to generate a detailed plan with subtasks
- Techniques for task decomposition:
  - **Chain of Thought** (Single-path reasoning)
  - **Tree of Thoughts** (Multi-path reasoning)

# Example comparison from Wang et al., 2023.



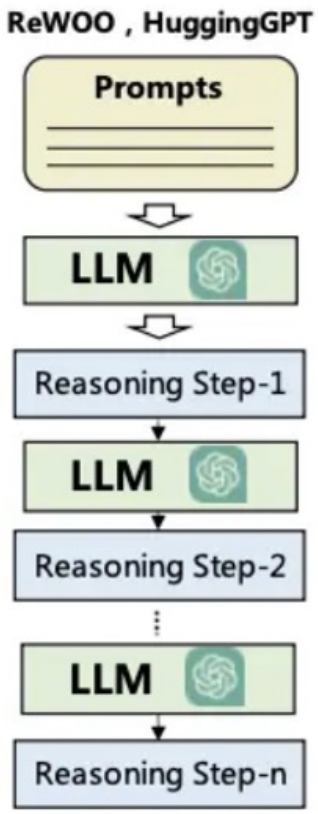
# CoT: Chain of Thought




Linear reasoning (single path)

- The model reasons step-by-step once
- Question: “If I have 3 apples and buy 2 more, how many do I have?”
  - Step 1: I have 3 apples
  - Step 2: I buy 2 more
  - Step 3: 3 + 2 = 5
  - Answer: 5
- Execution happens **implicitly inside each step**
  - No external actions (pure reasoning)
- If one step is wrong → everything is wrong

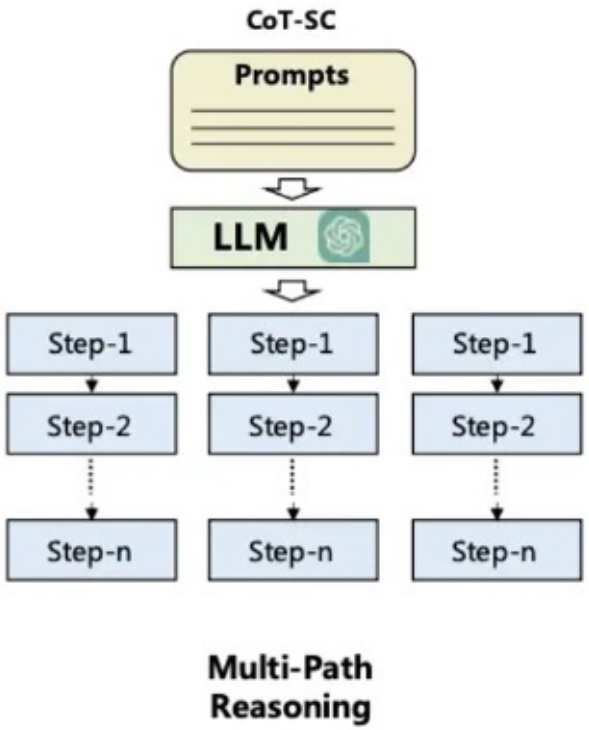
# ReWOO: Reasoning Wothout Observation




Planning + execution (no feedback)

- The model creates a full plan first, then executes it without changing it
- Question: “Find the CEO of Tesla and their age”
  - Plan: (1) Find CEO of Tesla; (2) Find age of the CEO
  - Execution: Step 1 → Elon Musk ;  Step 2 → 52 years old
  - Answer: Elon Musk, 52
- Execution happens after planning
  - Steps are executed sequentially (often via tools/APIs)
  - No feedback loop during execution
- If the plan is wrong → no correction

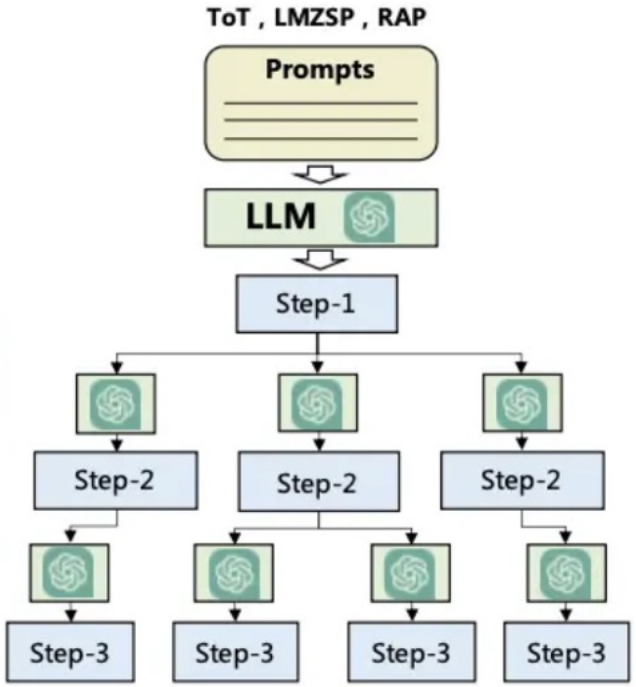
# CoT-SC: Chain of Thought – Self-Consistency




Multiple reasoning paths + voting

- The model generates multiple solutions and selects the most frequent one
- Question: “3 + 2 = ?”
  - Attempts:
    - Path 1 → 5
    - Path 2 → 6
    - Path 3 → 5
    - Path 4 → 5
  - Most frequent answer: 5
  - Answer: 5
- Execution happens **independently in each path**
  - No interaction between paths
  - Final “selection” acts as post-execution aggregation

# ToT: Tree of Thoughts / LMZSP: Language Model as a Zero-Shot Planner / RAP: Reasoning via Planning




Multi-path exploration with selection

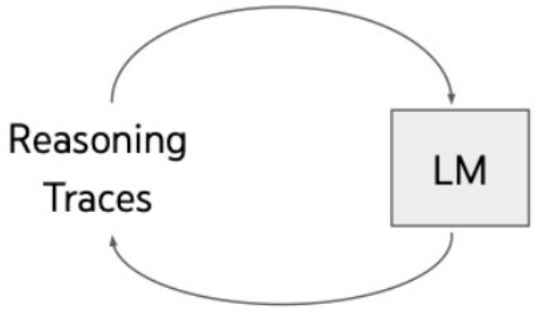
- The model explores different paths and selects the best ones during reasoning
- Question: “Reach 10 using 2, 3, 5”
  - Possible paths:
    - Path A: 2 + 3 + 5 = 10
    - Path B: 2 × 3 + 5 = 11
    - Path C: 5 + 5 = 10
  - Discard B, keep A and C
  - Answer: 10
- Execution is interleaved with reasoning
  - Partial solutions are evaluated during the process
  - Weak paths are pruned, strong ones expanded (feedback loop)

# Planning with Feedback

- Traditional planning modules lack feedback, making long-horizon planning difficult
- A **reflection** mechanism allows models to iteratively refine execution plans
  - Iteratively reflect and refine the execution plan based on past actions and observations
- This process corrects mistakes and improves final results
- Essential for complex real-world tasks where trial and error is key
- Two popular methods:
  - ReAct and Reflexion

# Reasoning OR acting

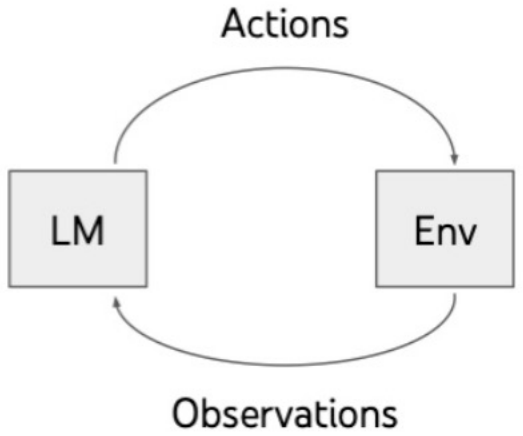
## CoT




Flexible and general to augment test-time compute

Lack of external knowledge and tools

## RAG/Retrieval/Code/Tool use




Retrieval
Search engine
Calculator
Weather API
Python
.....

Lack of reasoning

Flexible and general to augment knowledge, computation, feedback, etc.

# Acting without Reasoning

You are in the middle of a room. Looking quickly around you, you see a cabinet 6, a cabinet 1, a coffee machine 1, a countertop 3, a stove burner 1, and a toaster 1.
**Your task is to:** Put some pepper shaker on a drawer.

**Act only**

**Act 1:** Go to drawer 1
**Obs 1:** The drawer 1 is closed.

**Act 2:** Open drawer 1
**Obs 2:** You open the drawer 1. The drawer 1 is open. In it, you see a dishsponge 2, and a spoon 1.

**Act 3:** Go to sinkbasin 1
**Obs 3:** On the sinkbasin 1, you see a dishsponge 3, a spatula 1, and a spoon 2.

**Act 4:** Take peppershaker 1 from sinkbasin 1
**Obs 4:** Nothing happens.

**Act 5:** Take peppershaker 1 from sinkbasin 1
**Obs 5:** Nothing happens.
.....

X

**Cannot explore systematically or incorporate feedback**

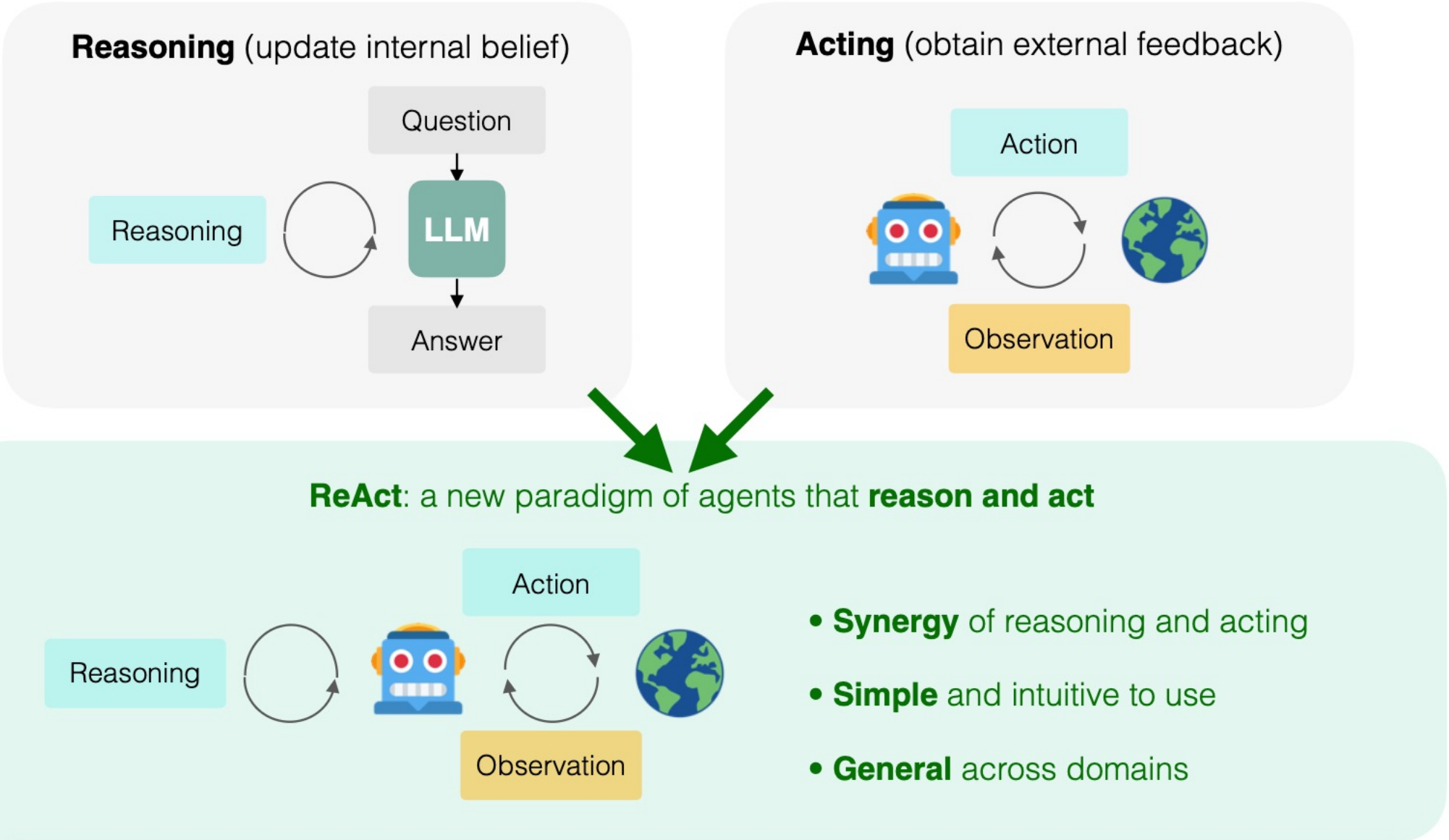
# ReAct: Interleaving Reasoning and Action

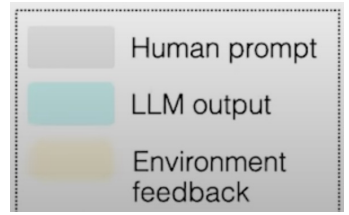
- **ReAct** (Yao et al., 2022) introduced a common agent pattern:
  - Interleave **reasoning**, **action**, and **observation**
- "Reasoning" includes:
  - Planning
  - Reflection
- The agent:
  - Explains its thinking
  - Takes actions
  - Analyzes observations
  - Stops when reflection determines the task is complete

```
Thought 1: …
Act 1: …
Observation 1: …

Thought 2: …
Act 2: …
Observation 2: …

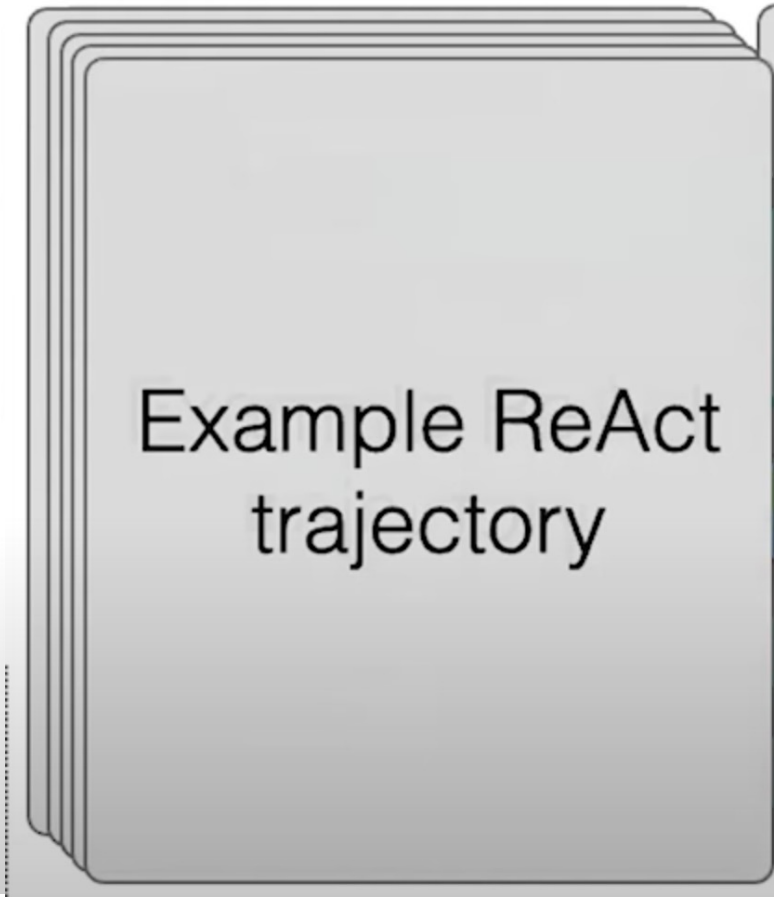
…
Act N: Finish
```


**Reasoning** (update internal belief)
Question
Reasoning
LLM
Answer
**Acting** (obtain external feedback)
Action
Observation
**ReAct**: a new paradigm of agents that **reason and act**
Action
Reasoning
Observation
• **Synergy** of reasoning and acting
• **Simple** and intuitive to use
• **General** across domains




ReAct supports

- One-shot prompting
- Few-shot prompting
- Fine-tuning






# Zero-shot ReAct prompt

You are an agent that answers questions by using two actions:
- search[query]: Google search the query. You can also use it to calculate math.
- finish[answer]: return the answer

Your generation should have the following format:
**Thought**: to analyze the observation and inform the action.
**Action**: your action

**Question**: If I have seven trillion dollars today, can I buy Apple, Nvidia, and Microsoft? If not, how much more money do I need?

LLM

(GPT-4)

**Thought**: I need to find the current market capitalization of Apple, Nvidia, and Microsoft to understand how much it would cost to buy all of them.
**Action**: search["current market capitalization of Apple, Nvidia, and Microsoft"]

**Observation**: "Nvidia's market cap stood at $1.832 trillion as of about 10 a.m. ET Wednesday, while Alphabet's was at $1.814 trillion, bringing Nvidia behind just Microsoft (MSFT) at $3.015 trillion and Apple (AAPL) at $2.836 trillion."



(Mar 2024)

**Thought**: Now that I have the market cap for all companies, let's add them together and see if $7 trillion is enough.
**Action**: search["1.832 trillion + 3.015 trillion + 2.836 trillion"]

**Observation**: 7683000000000

**Thought**: The amount of money needed in addition to the $7 trillion is $683 billion.
**Action**: finish["You need an additional $683 billion to be able to buy Apple, Nvidia, and Microsoft."]



LLM

(GPT-4)

**Thought**: I need to find the current market capitalization of Apple, Nvidia, and Microsoft to understand how much it would cost to buy all of them.
**Action**: search["current market capitalization of Apple, Nvidia, and Microsoft"]

**Observation**: nothing found! (My adversarial observation)

**Thought**: The search did not give any results. I will need to search for each company's market capitalization individually.
**Action**: search["current market capitalization of Apple"]

**Observation**: 'price': 180.75 (My adversarial observation)

**Thought:** This is the current price of Apple's stock, not its market capitalization. The market cap is calculated by multiplying the share price by the number of outstanding shares. Thus, I need to find the total number of Apple's outstanding shares to calculate its market cap.
**Action**: search["total number of Apple's outstanding shares"] ✓ (Continues to solve the task)

**Synergy:** acting support reasoning, reasoning guides acting

# ReAct Enables Systematic Exploration

You are in the middle of a room. Looking quickly around you, you see a cabinet 6, a cabinet 1, a coffee machine 1, a countertop 3, a stove burner 1, and a toaster 1.
**Your task is to:** Put some pepper shaker on a drawer.

**ReAct**

**Act 1:** Think[First I need to find a peppe shaker...more likely to appear in cabinets (1-6), countertops (1-3), ... ]
**Act 2:** Go to cabinet 1
**Obs 2:** On the cabinet 1, you see a vase 2.
(Here the agent goes to cabinet 1, 2, 3, 4, 5, 6, then countertop 1 and 2)

**Act 9:** Go to countertop 3
**Obs 9:** On the countertop 3, you see a apple 1, a bread 1, ..... a pepper shaker 1, and a vase 3.
**Act 10:** Take pepper shaker 1 from countertop 3
**Obs 10:** You pick up the pepper shaker 1 from the countertop 3.
**Act 11:** Think[Now I find a pepper shaker 1. Next, I need to put it in/on drawer 1.]
**Act 12:** Go to drawer 1
**Obs 12:** Drawer 1 is closed.
**Act 13:** Open drawer 1
**Obs 13:** You open Drawer 1 …
**Act 14:** Put pepper shaker 1 in/on drawer 1
**Obs 14:** You put pepper shaker 1 in/on the drawer 1.



## (1) Hotspot QA

**Question:** Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

### (1d) ReAct (Reason + Act)

**Thought 1:** I need to search Apple Remote and find the program it was originally designed to interact with.

**Act 1:** Search[Apple Remote]

**Obs 1:** The Apple Remote is a remote control introducduced in October 2005 by Apple … originally designed to control the Front Row media center program

**Thought 2:** Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

**Thought 3:** Apple Remote was originally designed to control Front Row (Front Row media center program. I need to search Front Row next and find

**Thought 4:** Front Row is not found. I need to search Front Row (software) , ..]

**Thought 4:** Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

**Act 4:** Finish[keyboard function keys]



# Reflection and Error Correction

- Even the best plans must be **continuously evaluated and adjusted**
- Reflection is:
  - **Not strictly necessary** for an agent to operate
  - **Necessary for an agent to succeed**
- Reflection increases:
  - Robustness
  - Task success rate
  - Ability to recover from mistakes
- Reflection and **error correction** go hand in hand:
  - Reflection uncovers errors
  - Error correction fixes them

# When and How Reflection Is Used

Reflection can occur at multiple points during a task:

- **After receiving the user query**
  - Evaluate whether the request is feasible
- **After initial plan generation**
  - Check if the plan makes sense
- **After each execution step**
  - Verify the agent is on the right track
- **After full execution**
  - Determine whether the task was accomplished

- Reflection can be implemented:
  - By the **same agent** using self-critique prompts
  - By a **separate component** (evaluator / scorer)
- In a **multi-agent setting**
  - One agent acts
  - Another evaluates outcomes

# Reflexion: Learning from Failure

- **Reflexion** (Shinn et al., 2023) extends ReAct by explicitly learning from errors
- Reflection is split into two roles:
  - **Evaluator**: assesses outcomes
  - **Self-reflection module**: analyzes what went wrong
- The agent reasons over its **trajectory** (plan + actions)
- After reflection:
  - A **new trajectory** is proposed
- Example outcomes:
  - Code fails test cases
  - Reflection identifies missing edge cases
  - Agent generates improved code

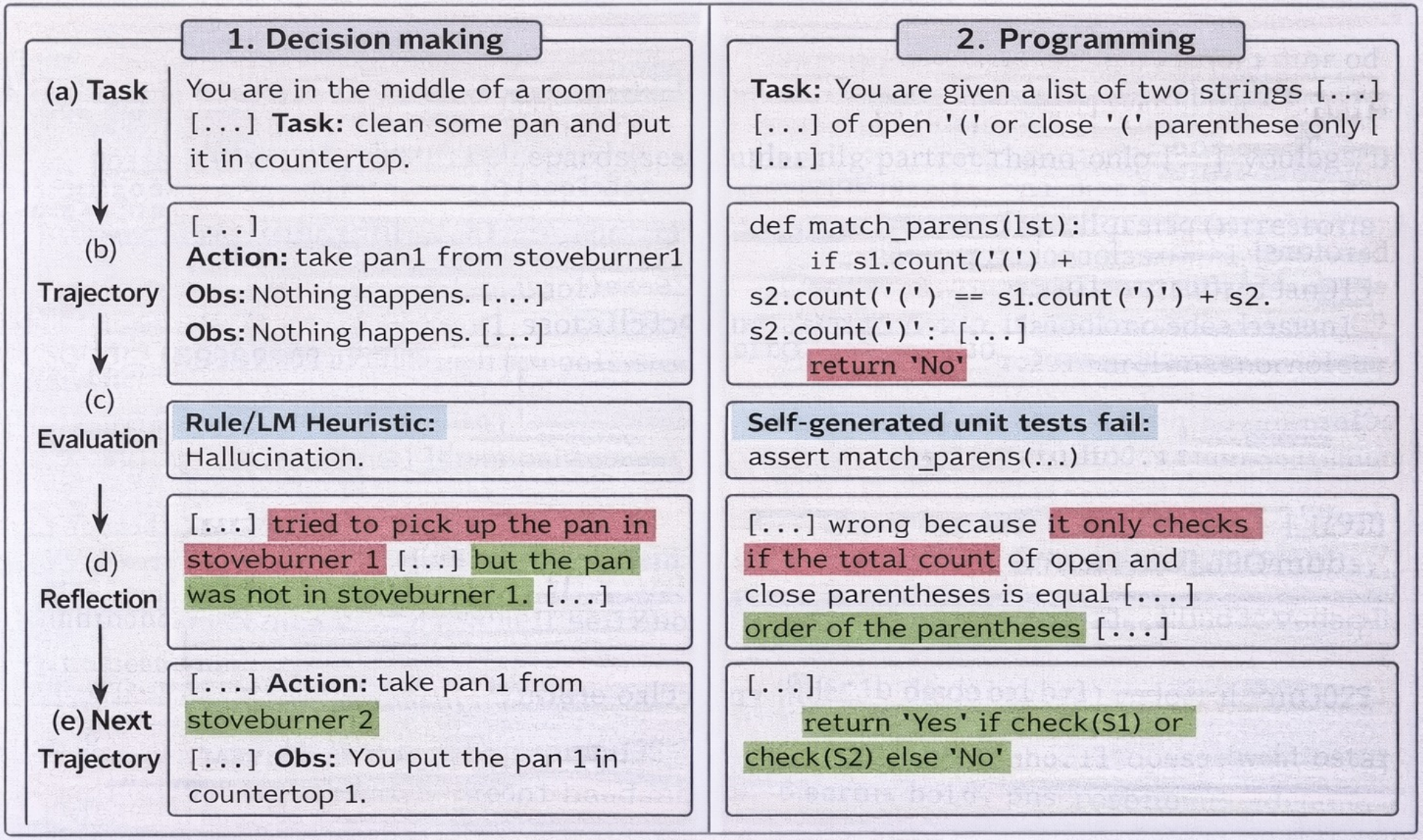



Figure 6-11. Examples of different orders in which a plan can be executed.

# Agent Failure Modes and Evaluation

- Agent evaluation requires analyzing **what fails**, **how often**, and **why** — across planning, tool use, and efficiency — not just whether the agent eventually succeeds
- Evaluation is about **detecting failures**
- As agent tasks become more complex the number of **possible failure points increases**
- In addition to standard AI failures, agents have **agent-specific failures** caused by:
  - planning
  - tool execution
  - efficiency
- Some failures are **easy to detect**, others are subtle

# How to Evaluate an Agent

- To evaluate an agent:
  - **identify its failure modes**
  - measure **how often each failure occurs**
- Evaluation focuses on **frequency and patterns**, not only success rate
- Existing benchmarks and tools include:
  - Berkeley Function Calling Leaderboard
  - AgentOps evaluation harness
  - TravelPlanner benchmark
- Different benchmarks target **different agent capabilities**

# Planning Failures: Tool Use Errors

Planning often fails due to **incorrect tool usage**

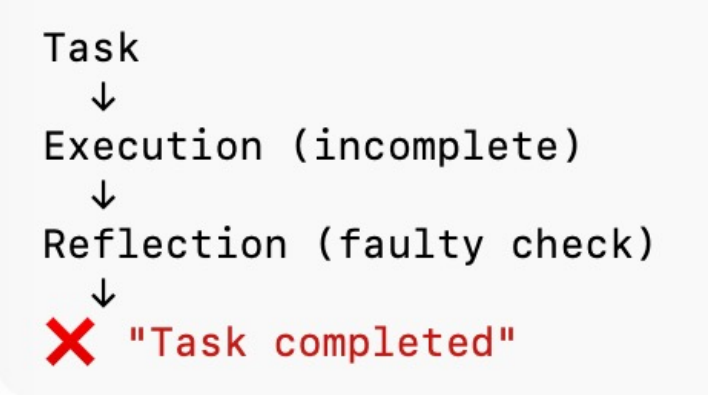
1. **Invalid tool**
   - Tool appears in the plan but is not in the tool inventory
   - Example: using bing_search when it is unavailable
2. **Valid tool, invalid parameters**
   - Correct tool, wrong number of parameters
   - Example: calling lbs_to_kg with two parameters instead of one
3. **Valid tool, incorrect parameter values**
   - Correct tool and schema, wrong values
   - Example: converting 100 lbs instead of the correct 120 lbs

# Planning Failures: Goal, Constraints, and Time

- **Goal failure**:
  - The plan does not solve the task
  - Or violates constraints (budget, location, duration)
  - Example:
    - Two-week trip with $5,000 budget
    - Plan exceeds budget or goes to the wrong city
- **Time is a critical but often overlooked constraint**
  - A correct solution delivered too late is still a failure

# Reflection Failures

- Reflection failures occur when:
  - the agent **believes the task is complete**, but it is not
  - Example:
    - Assign 50 people to 30 hotel rooms
    - Agent assigns only 40 people and claims success
- These failures are dangerous because:
  - execution may stop prematurely
  - errors are not surfaced automatically
- Compared to normal errors:
  - Normal error → visible (crash, inconsistency)
  - Reflection failure → silent error + false success




**“The model fails twice: it makes a mistake and fails to detect it”**

# Evaluating Planning Failures: Metrics

One evaluation strategy:

- Create a **planning dataset** of (task, tool inventory) pairs
- For each task, generate **K plans**

- Measure:
  - Fraction of valid plans
  - Average number of plans needed to get a valid one
  - Fraction of valid tool calls
  - Frequency of invalid tool usage
  - Frequency of invalid parameters
  - Frequency of incorrect parameter values
- Analyze outputs to identify **systematic weaknesses**

# Tool Failures and Efficiency

**Tool failures**

- Correct tool is used, but **output is wrong**
  - Image captioner returns incorrect description
  - SQL generator produces wrong query
- Failures can also arise from:
  - Missing tools (e.g., no internet access)
  - Translation layers between plans and execution
- Tools should be **tested independently**

**Efficiency**

- Even correct agents can be **inefficient**
- Key efficiency metrics:
  - Average number of steps per task
  - Average cost per task
  - Time per action
- Efficiency depends on the baseline:
  - Humans vs AI have **different notions of efficiency**